

trext

cert-manager: Past, Present, and Future

Speakers

Ash (Ashley) — Senior Software Engineer, Jetstack; cert-manager Maintainer
Jake — Platform Engineer, G-Research; cert-manager Maintainer

1) What cert-manager Is

cert-manager is a cloud-native Kubernetes operator that automates **X.509/TLS certificate** issuance, renewal, and distribution. It models PKI workflows with Kubernetes **CRDs** and integrates with a broad ecosystem of **Certificate Authorities (CAs)** and tools.

Core workflow (high level):

- You define **Issuers/ClusterIssuers** (how/where to get certs).
- You define **Certificates** (what identity you need).
- cert-manager talks to the CA, obtains the cert, and (by default) stores it in a **Kubernetes Secret** in the formats you need (PEM, **PKCS#12**, **JKS**, etc.).
- Your workloads mount/use the certs; cert-manager renews them in time.

For many users, the simplest path still works: **annotate an Ingress**, get a publicly trusted certificate (commonly via **ACME/Let's Encrypt**).

2) Where cert-manager Came From (Past)

- **kube-lego (Jetstack)** solved the “annotate Ingress → get Let's Encrypt cert” experience.
- cert-manager evolved that idea into a **Kubernetes-native** operator with **CRDs** for requests, challenges, and state.
- As adoption grew (included in several Kubernetes distributions), the team stabilized the API with a **v1 CRD** commitment: no breaking changes to that surface.

The project subsequently entered **CNCF incubation**, supported by the community, sponsors, and contributors across organizations.

3) How It Works (Present)

3.1 CRD Model

- **Issuer / ClusterIssuer:** configuration + auth for a CA (e.g., **Let's Encrypt/ACME**, **HashiCorp Vault**, **Venafi TPP**, cloud CAs, and many **external issuers**).
- **Certificate:** desired identities (DNS, IP, URI, SANs), key parameters, usages, duration/renewal policy.
- **Secret (default):** cert/key storage for workloads (multiple keystore formats supported).

3.2 Security Posture

cert-manager is **security-critical infrastructure**:

- **Supply chain:** container images are **cosign**-signed (Sigstore). The project follows **SLSA** practices (self-assessed: $\geq$ level 2, pushing toward 3).
- **Policy:** a built-in **approver/policy engine** lets you restrict who can issue what (e.g., only certain service accounts/namespaces may request production DNS names, enforce key sizes/algorithms).
- **Key rotation:** strongly encouraged on reissue (opt-out available for legacy apps, albeit discouraged).
- **Short-lived certs:** supported to reduce blast radius; frequent, automated renewal.

3.3 Integrations & Ecosystem

- **Service meshes:** can replace Citadel in **Istio**; tight integration with **Linkerd**.
- **Kubernetes Gateway API:** first-class support.
- **Observability:** **Prometheus** metrics for fleet-scale visibility (alert on pending/late renewals, failures).
- **CSI Drivers (two):**
 - **TLS CSI Driver:** mounts certs to pods while keeping **private keys in memory** (not persisted in the API).
 - **SPIFFE X.509 CSI Driver:** issues SPIFFE-compatible identities for mTLS-centric stacks.

Many enterprises prefer the CSI approach to avoid storing private keys in Kubernetes Secrets.

4) The Trust Problem (Beyond “Getting a Cert”)

Obtaining a certificate is **only half** of the story. Systems must **distribute and enforce trust** (which CAs/roots are acceptable, how bundles are curated/rotated, how trust differs across environments).

The project is exploring solutions (e.g., **Trust Manager**) to help manage **trust stores** and policy consistently across clusters and workloads.

5) Roadmap & Future Directions

- **Deeper integrations** across the CNCF landscape (service meshes, data planes, identity systems).
- **SPIFFE/SPIRE**: expand beyond basic SPIFFE support toward **federation** (join cert-manager and SPIRE trust domains).
- **Developer experience**: faster, more reliable CI; contributor-friendly docs, **GSoC** and **Season of Docs** participation.
- **Public roadmap**: maintained in the main repo (signals intent and invites discussion/+.1s).

Timelines are aspirational; the project prioritizes quality, security, and maintainability.

6) Operations Guidance

Area	Default / Legacy	Recommended / Modern
Auth to CA	Static tokens in Secrets	Workload identity (K8s SA tokens, OIDC) or SPIFFE
Storage	Kubernetes Secrets	CSI in-memory mounts (no API persistence of keys)
Lifetimes	Longer	Short-lived certs with aggressive auto-renew
Policy	Ad-hoc	Approver policy (DNS scope, requester, key strength)
Key Rotation	Optional	Rotate on every reissue (unless truly impossible)
Observability	Manual checks	Prometheus alerts on renewal windows/failures

7) Community & How to Get Involved

The project thrives on **contributors**—code, docs, policy examples, workshops, tutorials, issue triage, Slack Q&A, and more.

- **Meetings**: public standups (EU mornings) and a US-friendly bi-weekly call.
- **Slack**: CNCF `#cert-manager` channel.
- **Repo**: contribution guide, policies, and roadmap in **cert-manager/cert-manager**.

- **Events & Booth:** live demos (including printed X.509 “souvenir” certs), Q&A, and collaboration chats.

“If your project needs TLS or identity, let’s integrate—blog posts, workshops, or joint talks are very welcome.”

8) Summary

- **Past:** From kube-lego’s simple Ingress automation to a stable, widely adopted **CNCF** project.
- **Present:** A **modular, secure, policy-aware** operator with broad issuer and mesh integrations, plus CSI drivers for secret-averse environments.
- **Future:** Stronger **trust distribution**, **SPIFFE/SPIRE federation**, expanding **CNCF integrations**, and an even smoother contributor experience.

Bottom line: cert-manager makes Kubernetes TLS **boring**—automated, observable, and secure—so teams can focus on building systems, not babysitting certificates.